Working with Data and Cache Storage in Drupal

Drupal is a popular and widely used content management system (CMS). This article exposes the reader to MySQL database solutions for Drupal, along with others like PostgreSQL and SQLite, and follows up with caching mechanisms.

One common belief that I encounter among developers is that if they make Drupal their career, they are bound to get stuck with one technology and only the LAMP stack. I felt that way too, when I started out as a Drupal developer seven years ago. But the varied projects I worked on soon debunked that myth.

Yes, it is definitely true that the LAMP stack is not the only thing that you get to work on with when you are on Drupal. You need to be more than a simple PHP developer to explore the wild, wild ways of the Drupal world. A small glimpse into the database side of this amazing CMS platform will reveal this.

Drupal loves MySQL

MySQL is ideal as a primary database for Drupal, because the Drupal core is built on the assumption that it will be deployed on a finely tuned LAMP stack. Even the caching layer in Drupal stores its data in MySQL tables. Drupal's database abstraction layer makes it easy for the underlying DB to be replaced with any other, provided that there is a driver implemented for it and a PHP extension that connects the PHP layer to the database engine. Drupal's database abstraction layer uses the PDO's (PHP data object's) database API and hence derives much of its semantics and syntax from it. This layer also puts in a lot of security checks, best practices and optimisations. Hence, it must never be bypassed in any customisation efforts. Drupal's database implementation also supports a master-slave configuration for higher performance.

A lot of hosting providers that support CMSs like Drupal go in for variants of MySQL, like Percona Server and MariaDB, as well as the enterprise version of Oracle MySQL to have additional features or performance benefits. Acquia Cloud uses the Percona Server as its database of choice.

So what if we need something other than MySQL? The options that Drupal provides as a part of its core are PostgreSQL and SQLite. Now why would we use these, if MySQL works really well, out-of-the-box? Anyway, here are some options.

PostgreSQL

PostgreSQL has a very good and complete ACID implementation, and various algorithms built in for complex SQL operations like JOINS, UNIONS and such. So, if your Drupal applications are heavy in terms of data structure, you could use PostgreSQL. If this database is part of a bigger system, where data is shared amongst applications like a BI tool, it would be good to consolidate to a database that supports all your applications, including Drupal. Support for PostgreSQL is part of Drupal 7 Core.

SQLite

This lightweight database extends its properties to your Drupal application. SQLite is a serverless single file database engine. It is a great option for small sites with low to medium traffic. The overheads of maintaining a MySQL or PostgreSQL server have been removed, and the infrastructure required to run the show is also less. Drupal Core provides SQLite as an option, as a part of its core features. This database is ideal for creating microsites, advertisements, internal information sites and such, in educational institutions, NGOs, etc.

MongoDB

The Drupal community has contributed an integration of Drupal's database layer with MongoDB for special use cases, such as hiving off the cache tables to a flat file structure like what MongoDB has. MongoDB is also used for field storage, session, watchdog, queue and block, to enable faster reads and take advantage of the flat file